

**INSTITUTO
FEDERAL**
Santa Catarina

Orientação a Objetos em JavaScript (parte 1)

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas

cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono



Objetos

o que são

- **coleções não ordenadas de propriedades, cada uma com um nome e um valor**
- propriedades podem ser adicionadas e excluídas
- nomes geralmente são strings
- valores podem ser primitivos ou objetos

exemplos

```
let vazio = {};
```

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan",  
  },  
};
```



Objetos

criando objetos

- como objetos literais

exemplos

```
let vazio = {};
```

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan",  
  },  
};
```



Objetos

criando objetos

- como objetos literais
- utilizando um construtor

exemplos

```
let o = new Object();  
// Cria um objeto vazio: o mesmo que {}.  
  
let a = new Array();  
// Cria um array vazio: o mesmo que [].  
  
let d = new Date();  
// Cria um objeto Date representando o horário atual.  
  
let r = new Map();  
// Cria um objeto Map para mapeamento de chave/valor.
```



Objetos

protótipos

- quase todos os objetos JS têm um protótipo (prototype) associado
- objetos herdam suas propriedades do protótipo
- objetos literais herdam de `Object.prototype`
- objetos criados com construtores herdam do protótipo do construtor



Objetos

criando objetos

- como objetos literais
- utilizando um construtor
- com `Object.create()`

exemplos

```
let objeto = Object.create({ x: 1, y: 2 });
```

```
let vazio = Object.create(Object.prototype);  
// vazio é como {} ou new Object().
```

```
let nulo = Object.create(null);
```



Objetos

herança

- objeto herda as propriedades de seu protótipo

exemplos

```
let o = {};  
o.x = 1;
```

```
let p = Object.create(o);  
p.y = 2;
```

```
let q = Object.create(p);  
q.z = 3;
```

```
q.x + q.y; // 3
```



Objetos

herança

- objeto herda as propriedades de seu protótipo
- **herança dinâmica**

exemplos

```
let objeto = {};  
let outroObjeto = Object.create(objeto);
```

```
objeto.x = 1;  
outroObjeto.x; // 1
```

```
objeto.x = 2;  
outroObjeto.x; // 2
```



Objetos

herança

- objeto herda as propriedades de seu protótipo
- herança dinâmica
- **propriedades herdadas podem ser sobrescritas**

exemplos

```
let circulo = { raio: 1 };  
let c = Object.create(circulo);
```

```
c.raio = 2;  
c.borda = 0.1;
```

```
circulo.raio; // 1
```



Classes

definição

- um conjunto de objetos que herdaram propriedades do mesmo protótipo

exemplos

```
let circulo = { raio: 1 };  
let pessoa = { nome: "José", idade: 35 };  
let carro = { modelo: "Ford", ano: 2022 };
```



Classes

criando objetos de uma classe

■ função fábrica

- retorna um objeto
- classe é definida pelo protótipo
- atributos na função não são compartilhados entre as instâncias

exemplos

```
function retangulo(largura, altura) {  
  let r = Object.create(retangulo.metodos);  
  r.largura = largura;  
  r.altura = altura;  
  return r;  
}  
  
retangulo.metodos = {  
  area: function() {  
    return this.largura * this.altura;  
  },  
  perimetro: function() {  
    return 2 * (this.largura + this.altura);  
  }  
}
```



Classes

criando objetos de uma classe

- função fábrica
- **construtor**
 - função própria para a inicialização de novos objetos
 - utilizam o atributo prototype
 - todos os objetos criados com o mesmo construtor pertencem à mesma classe
 - construtores diferentes que possuem o mesmo protótipo criam objetos da mesma classe

exemplos

```
function Retangulo(largura, altura) {  
    this.largura = largura;  
    this.altura = altura;  
}  
  
Retangulo.prototype = {  
    area: function() {  
        return this.largura * this.altura;  
    },  
    perimetro: function() {  
        return 2 * (this.largura + this.altura);  
    }  
}  
  
let ret = new Retangulo(2, 5);
```



Classes

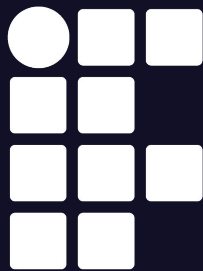
criando objetos de uma classe

- função fábrica
- construtor
- **classe (class)**
 - sintaxe própria para classes a partir do ES6
 - construtor definido com constructor
 - não suporta a definição de propriedades com pares nome/valor
 - não é "içada" para o início do documento

exemplos

```
class Retangulo {  
  constructor(largura, altura) {  
    this.largura = largura;  
    this.altura = altura;  
  }  
  area() {  
    return this.largura * this.altura;  
  }  
  perimetro() {  
    return 2 * (this.largura + this.altura);  
  }  
}  
  
let ret = new Retangulo(2, 5);
```





**INSTITUTO
FEDERAL**
Santa Catarina

Orientação a Objetos em JavaScript (parte 1)

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br

